CATÁLOGO PARA INGENIERÍA DE SOFTWARE

Patrones de Microservicios

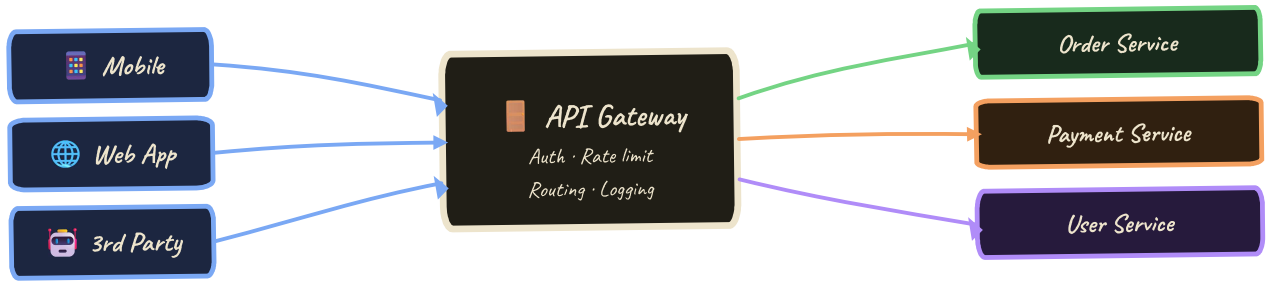
Soluciones probadas a los problemas más comunes cuando divides tu sistema en servicios independientes

Comunicación · Resiliencia · Datos · Deployment · Migración

Comunicación y Enrutamiento

API Gateway

Un **punto de entrada único** para todos los clientes. En lugar de que cada cliente llame a cada servicio directamente, el Gateway centraliza: autenticación, rate limiting, logging y enrutamiento.



Kong · AWS API GW · Nginx · Traefik

Service Discovery

Los servicios no usan IPs fijas. Se **registran en un "directorio"** al arrancar y lo consultan para encontrarse.

BFF – Backend for Frontend

Un **backend especializado por tipo de cliente.** En lugar de un API genérica, tienes una para mobile y otra para web, cada una con

CLIENT-SIDE

El cliente consulta el registro y elige la instancia (Ribbon, Eureka)

SERVER-SIDE

El load balancer consulta el registro (AWS ALB, Consul)

Consul Eureka etcd

los datos que ese cliente necesita exactamente.

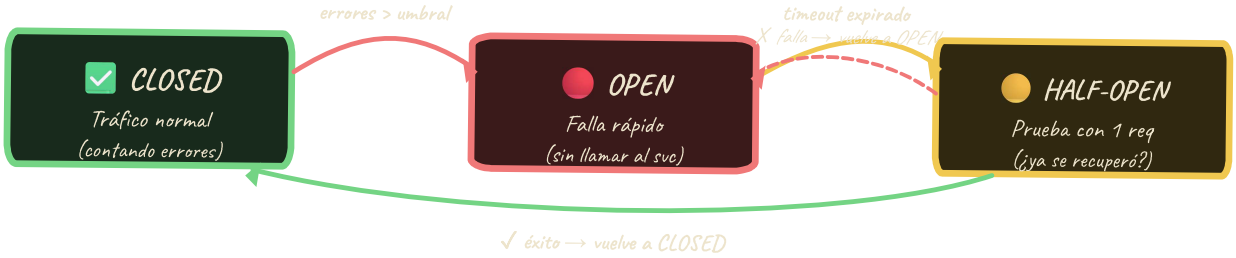
Evita que el cliente haga múltiples llamadas o reciba datos innecesarios.

GraphQL REST gRPC

Resiliencia y Tolerancia a Fallos

Circuit Breaker

Cuando un servicio falla repetidamente, **"corta el circuito"** para evitar que los fallos en cascada tumben todo el sistema. Como el breaker eléctrico de tu casa.



Resilience4j Hystrix Polly (.NET)

Bulkhead (Mamparo)

Como los compartimentos estancos de un barco: **aísla recursos por servicio** (thread pools, conexiones). Si un servicio colapsa, no consume los recursos de los demás.

Pool A

Pool B

Pool C ✓

Retry + Timeout

Retry: reintenta automáticamente las llamadas fallidas. Usa backoff exponencial para no saturar el servicio caído.

Timeout: no esperes indefinidamente. Define un tiempo máximo y falla rápido si no hay respuesta.

10 threads
Order Svc

10 threads
Payment Svc

10 threads
User Svc

Pool B colapsa → Pool A y C siguen funcionando sin problema.

1er intento → falla

Espera 1s → 2do intento → falla

Espera 2s → 3er intento → falla

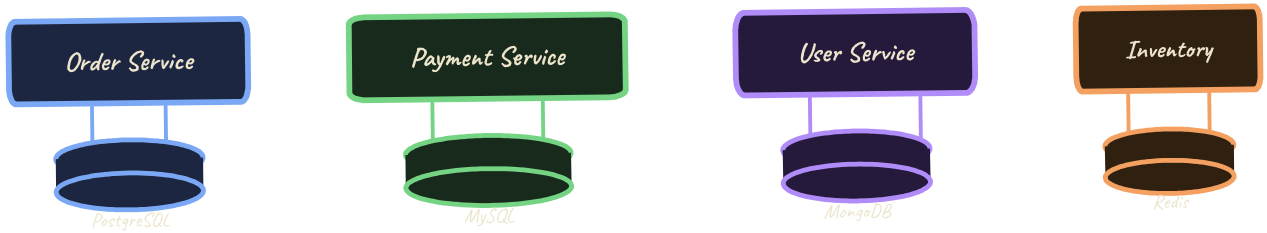
Espera 4s → 4to intento → ✓

⚠️ Jitter: agrega aleatoriedad al backoff para evitar que todos los clientes reintenten al mismo tiempo.

Gestión de Datos Distribuidos

Database per Service

Cada microservicio **posee su propia base de datos**, inaccesible directamente para otros servicios. Nadie hace JOIN entre BDs ajenas — los datos se comparten vía API o eventos.



🚫 Jamás: SELECT * FROM payment_db.orders JOIN order_db.items — eso acopla los servicios a nivel de BD.

CQRS — Command Query Responsibility Segregation

Separa el **modelo de escritura** (Commands) del **modelo de lectura** (Queries). Cada lado puede escalar de forma independiente y usar la BD más apropiada.





Event Sourcing | MediatR (.NET) | Axon

📜 Event Sourcing

En lugar de guardar el **estado actual** de un objeto, guardas la **secuencia de eventos** que lo llevaron a ese estado. El estado actual se reconstruye reproduciendo los eventos.

Tradicional ✗

orders table:
status = "shipped"
total = $150
(¿qué pasó antes?)

vs

Event Sourcing ✓

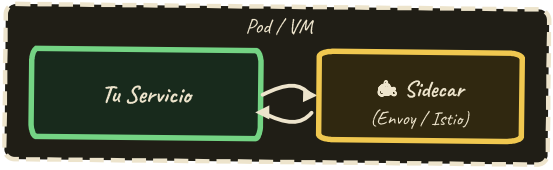
1. OrderCreated
2. ItemAdded($150)
3. PaymentDone
4. OrderShipped

💡 Beneficio extra: puedes hacer time travel — reconstruir el estado en cualquier momento del pasado.

🔧 Deployment y Migración

🏍 Sidecar

Despliega un **proceso auxiliar junto al servicio principal** en el mismo pod/VM. El sidecar maneja: observabilidad, seguridad TLS, circuit breaking — sin modificar el código del servicio.



🌳 Strangler Fig

Migra un **monolito a microservicios incrementalmente**. Como la higuera estranguladora: el nuevo sistema crece alrededor del viejo hasta reemplazarlo completamente, sin big bang.

Fase 1	100% Monolito
Fase 2	Facade ⟶ extrae módulos uno a uno
Fase N	100% Microservicios ✓

Envoy Linkerd Dapr

Fase IV 100% microservicios ✓

🔑 Clave: el facade (API Gateway) decide si la petición va al monolito viejo o al nuevo microservicio.

📋 REFERENCIA RÁPIDA

PATRÓN	PROBLEMA QUE RESUELVE	CATEGORÍA
API Gateway	Punto de entrada único, cross-cutting concerns	Comunicación
Service Discovery	¿Cómo se encuentran los servicios entre sí?	Comunicación
Circuit Breaker	Fallos en cascada cuando un servicio cae	Resiliencia
Bulkhead	Un servicio consume todos los recursos disponibles	Resiliencia
DB per Service	Acoplamiento de datos entre servicios	Datos
CQRS	Modelos de lectura/escritura con necesidades diferentes	Datos
Event Sourcing	Audit log, time travel, estado reconstruible	Datos
SAGA	Transacciones distribuidas entre servicios	Datos
Sidecar	Cross-cutting concerns sin tocar el código del servicio	Infra
Strangler Fig	Migrar un monolito a microservicios sin big bang	Migración

🧩 Estos patrones se **combinan**: API Gateway + Circuit Breaker + Sidecar es una pila muy común en producción.